TD6 : Programmation Orientée Objet (POO)

IN323 - Programmation en langage C++

Elisa HENRION-GUENEAU
IPSA - Site Toulouse
AERO3 – Année 2025/2026

Exercice 1 : Extension de la classe `Point`

On souhaite enrichir la classe `Point` définie dans le cours pour lui permettre d'interagir avec d'autres instances.

1. Créer la classe `Point` avec les attributs x et y privés.
2. Ajouter une méthode `double distance(Point &autre)` qui retourne la distance euclidienne entre le point courant (`this`) et un autre point passé en paramètre.

   Rappel : Distance d entre deux points A et B, $d = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$.
   Inclure la bibliothèque `<cmath>` pour utiliser les fonctions `sqrt()` et `pow()`.
3. Ajouter une méthode `Point milieu(Point &autre)` qui renvoie un nouvel objet `Point` situé exactement au milieu du segment formé par le point courant et le point `autre`.
4. Dans le main, créer deux points $P1(0, 0)$ et $P2(3, 4)$. Afficher la distance entre les deux (qui doit être 5) et les coordonnées du point milieu (à l'aide de la méthode `afficher`).

Exercice 2 : La classe `Rectangle`

1. Créer une classe `Rectangle` avec les attributs suivants :
   — `Point hautGauche`
   — `int longueur, largeur`
   — `static int nbRectangles` (un compteur pour suivre le nombre d'instances actives)
2. Incrémenter/décrémenter `nbRectangles` dans le constructeur/destructeur.
3. Créer un constructeur privé `Rectangle(Point &p1, Point &p2)` qui calcule la longueur et la largeur à partir de deux points opposés.
   Remarquer que la classe Rectangle ne peut pas accéder aux coordonnées x et y de la classe Point car elles sont privées. Pour résoudre ce problème, ajouter les méthodes `int getX()` et `int getY()` à la classe `Point` (méthodes qui renvoient respectivement x et y)
4. Créer une méthode statique publique `Rectangle creerDepuisPoints(Point &p1, Point &p2)`
   — Doit vérifier que les points permettent de créer un rectangle valide (longueur > 0 et largeur > 0).
   — Si c'est valide, elle retourne un objet Rectangle via le constructeur privé.
   — Sinon, elle peut afficher un message d'erreur et retourner un rectangle par défaut.

   Rq : Une méthode statique s'appelle via le nom de la classe (`Classe::Methode`) et peut accéder aux membres privés de sa propre classe.
5. Créer la méthode `void afficher()` : affiche les coordonnées du point haut-gauche, la longueur et la largeur
6. Créer la méthode `void translation(int dx, int dy)` : déplace le rectangle en modifiant son point haut gauche.

Exercice 3 : Gestion d'une file d'attente (Mémoire dynamique)

1. **Classe `Client`** :
   — Créer la classe `Client` avec les attributs privés `nom`, `prenom` et `id` (un entier).
   — Ajouter un membre `static int compteur` pour attribuer un identifiant unique auto-incrémenté à chaque nouveau client.

— Implémenter le constructeur et une méthode `void afficher()`.

2. **Classe FileDAttente** :
   — Créer la classe `FileDAttente` avec les attributs `Client *tab` (pointeur vers un tableau), `int tailleCourante` et `int capacite`.
   — Implémenter le constructeur qui prend la `capacite` en paramètre et alloue dynamiquement un tableau `tab` de taille `capacite`.
   — Implémenter le destructeur pour libérer la mémoire.
3. **Méthodes de gestion** :
   — Écrire la méthode `void ajouter(Client &c)` pour ajouter un client à la fin de la file (si le tableau n'est pas plein).
   — Écrire la méthode `void suivant()` pour sortir le premier client de la file et décaler tous les autres clients d'une case vers la gauche.
   — Écrire la méthode `void passeDevant(int idRecherche)` pour chercher un client par son identifiant et l'échanger avec le client qui se trouve juste devant lui.
4. **Dépassement de capacité** :
   — Modifier la méthode `ajouter` pour que, si le tableau est plein, on alloue un nouveau tableau deux fois plus grand, on y copie les clients actuels, on libère l'ancien tableau et on mette à jour le pointeur `tab`.